CSU33D06 Report: Analysis of Headscale

A Self-Hosted Tailscale Control Server

Xiaoyu SHEN
25369847

April 2026

1 Introduction

Headscale is an open-source, self-hosted re-implementation of the Tailscale control plane, written in Go. It enables individuals and organisations to operate a private WireGuard-based mesh network without depending on Tailscale's commercial infrastructure [1]. By the date of this report, the project has over 36,177 GitHub stars, has evolved into widely used infrastructure with a large user community [2].

Headscale is maintained by Kristoffer Dalby and Juan Font [3]. As documented in the maintainer's own account [4], the project began as a small learning experiment intended to understand how Tailscale worked. This trajectory raises a central question for our analysis: **When an organically grown open-source project scales beyond its original scope without a proportional increase in maintainers or governance structure, what forms of technical and organisational debt emerge, and how are they observable in the codebase?**

To answer this question, this report adopts a data-driven, multi-metric analysis framework that traces the evolution of the project across its lifecycle. Specifically, four complementary metrics are employed to capture different dimensions of technical and organisational debt: **hotspot concentration** to identify structural bottlenecks in the codebase, **knowledge concentration** to evaluate the distribution of critical expertise among contributors, **coordination latency** to measure the cost of cross-author collaboration, and **mean time to repair** to assess maintenance efficiency and user-facing quality outcomes. In addition, five key stable releases were selected to represent distinct stages in the project's growth trajectory, which will make the analysis more granular and insightful.

2 Why This Project

TCD School of Computer Science and Statistics uses Tailscale as its VPN solution [5], which intrigued me to explore its details. Headscale stood out as a compelling case study for several reasons:

- **Popularity and Activity:** With over 36k stars and 1.5k forks, Headscale has a vibrant community and active development, providing rich data for analysis [2].
- **Technical Complexity:** As a control plane for a mesh VPN, it involves complex networking concepts, state management, and security considerations, making it an interesting subject for software architecture analysis.
- **Organic Growth:** The project evolved from a learning experiment to a widely used tool without a formal governance structure, presenting an opportunity to study the implications of such growth on technical debt and maintainability.
- **My Interest:** As a user of Tailscale and a student who wants to pore over cybersecurity and networking, I want to improve my understanding of this field by analyzing an influential engineering project.

3 Stakeholders

Following Rozanski and Woods [6], the stakeholder breakdown below classifies the primary stakeholders identified through analysis of GitHub logs and the keynote address delivered by the project's lead maintainer at FOSDEM 2026 [4]. According to this address, the project originated with Juan Font in 2020 during the early COVID-19 period, with Kristoffer Dalby joining as a core contributor in 2021. Following the project's rapid growth in community adoption, Dalby was subsequently hired by Tailscale in 2022, with half his contracted working time formally dedicated to Headscale's continued development.

Unlike projects driven by well-organised teams, structured architecture, and clearly defined roles, Headscale follows a more informal development model. Consequently, its stakeholders' roles are usually fluid and overlapping, and certain roles within Rozanski and Woods' framework [6] remain unfilled or ambiguous. The following stakeholder breakdown and Context View diagram therefore consolidate similar roles and introduce selected "external" components to better capture the project's unique dynamics and interactions.

Entities labelled as External Entity fall outside Rozanski and Woods' stakeholder taxonomy but are included to capture interactions that are material to the system's context and governance dynamics.

3.1 Juan Font

Role: Developer, Maintainer, Tester
Reason: Founder of Headscale and co-maintainer.

3.2 Kristoffer Dalby

Role: Developer, Maintainer, Tester
Reason: Lead maintainer of the project.

3.3 Florian Preinstorfer

Role: Communicator, Tester
Reason: Kristoffer described him as an "excellent documentation writer" who contributes heavily to testing; this is further supported by git log analysis showing substantial documentation and testing contributions.

3.4 Open-Source Contributors

Role: Developers
Reason: Contribute code through GitHub.

3.5 Tailscale Inc.

Role: Supplier
Reason: Funds a portion of Kristoffer Dalby's time by employing him, and supplies the protocol and ecosystem upon which Headscale depends. Tailscale engineers maintain active collaboration with the Headscale maintainer, including providing early access to protocol design documents ahead of client releases, ensuring implementation compatibility.

3.6 Home-Lab or Self-Hoster Users

Role: Users, Testers, System Administrators, Production Engineers
Reason: End users who self-host and maintain private deployments.

3.7 Enterprise or Corporate Users

Role: Users, System Administrators, Production Engineers
Reason: Some home lab users bring headscale to their organisations, and some companies adopt it to avoid vendor lock-in.

3.8 Discord Server Members

Role: Support Staff
Reason: Provide community support and discussion in the Discord channel operated by Kristoffer Dalby.

3.9 GitHub (juanfont/headscale)

Role: External Entity
Reason: Primary platform for source code, issues, pull requests, and release management.

3.10 OSS Project Adopters

Role: External Entity
Reason: View Headscale as a fallback to reduce vendor lock-in risk. Headscale makes them dare to use Tailscale SaaS.

3.11 Skeptical Community Members

Role: External Entity
Reason: Concerned about Tailscale's potential embrace-extend-extinguish trajectory and suspect Kristoffer closing issues to protect Tailscale's interest.

3.12 Tailscale Client (OSS)

Role: Supplier
Reason: Headscale depends on the debug function or debug feature of the Tailscale client, which allows kind of overriding the control server.

3.13 IonScale / WireFire

Role: External Entity
Reason: Alternative open-source implementations of the Tailscale control server protocol. IonScale explicitly positions itself as a lightweight alternative currently in beta, in contrast to Headscale's goal of full feature parity with Tailscale SaaS. Both projects validate the viability of the open protocol.

3.14 Tailscale SaaS

Role: External Entity, Alternative
Reason: A proprietary hosted VPN service operated by Tailscale Inc., implementing the same protocol as Headscale. The two are complementary: Headscale serves as an open-source self-hosted alternative.

4 Context View

The context view in Figure 1 defines the system boundary of Headscale and its principal external interactions.

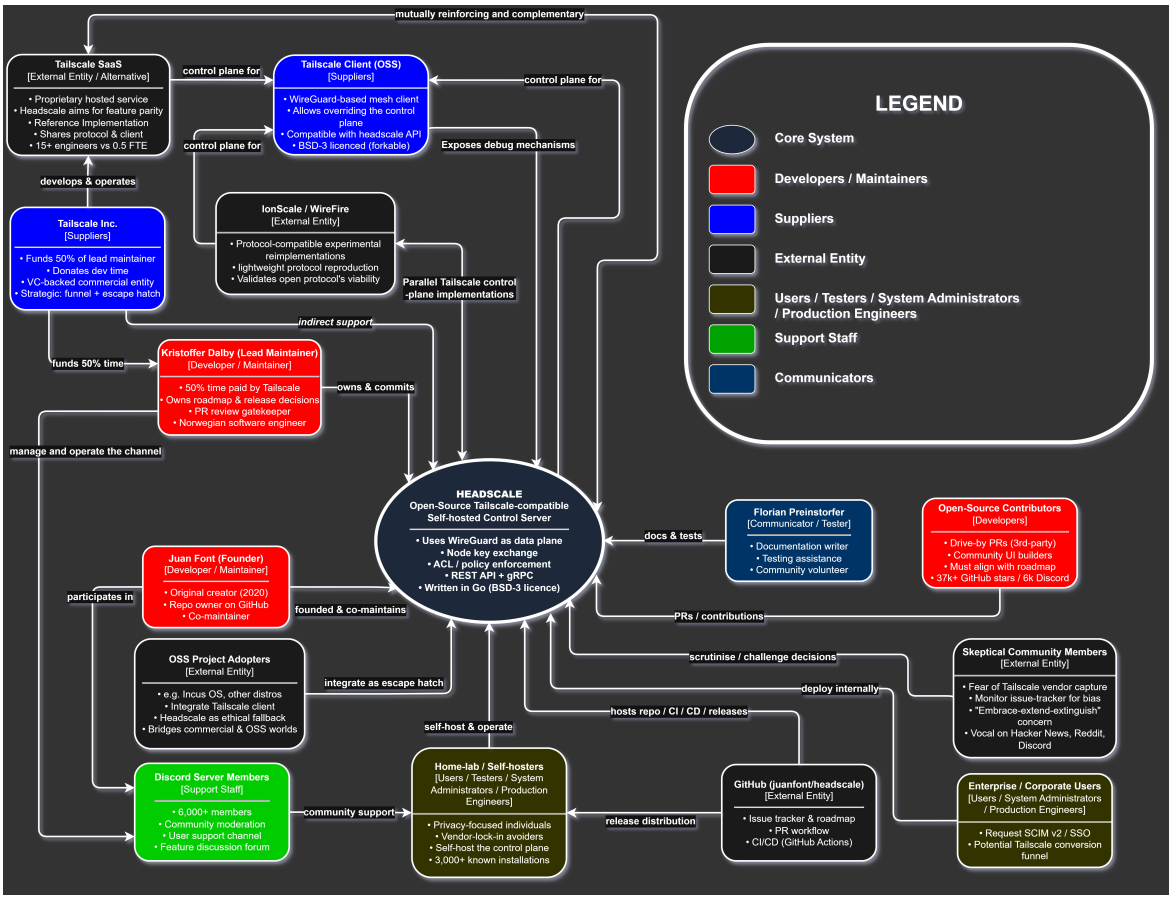


Figure 1: System context view of Headscale and its external interactions.

Tailscale Inc. <–> Headscale The most structurally significant external relationship in Headscale's context is the funding arrangement between Tailscale Inc. and its lead maintainer. As Dalby described at FOSDEM 2026 [4], Tailscale Inc. employs him as a software engineer with 50

Tailscale Client –> Headscale Headscale's operation depends on a debug feature in the open-source Tailscale client that allows the control server endpoint to be overridden. While Dalby acknowledged this could theoretically be removed [4], he noted that the client is BSD-3 licensed and therefore forkable, providing a meaningful fallback. This dependency is therefore better characterised as a protocol-compatibility constraint than a critical vulnerability: so long as the open-source client remains forkable, the community retains the ability to maintain a compatible client independently.

Tailscale SaaS <–> Headscale Headscale positions itself as complementary to rather than competitive with Tailscale SaaS, implementing the same protocol and aiming for feature parity with the hosted service. However, the relationship is fundamentally asymmetric: Tailscale SaaS is developed by a team of over 15 engineers, while Headscale is maintained by the equivalent of 0.5 full-time employees [4]. Dalby described the practical implication clearly — full feature parity is not a realistic near-term goal, and the gap is not merely a matter of velocity but of structural capacity. Tailscale SaaS thus acts as both the reference implementation and an implicit ceiling on Headscale's roadmap ambitions. The protocol design documents shared by Tailscale engineers in advance of client releases partially mitigate this asymmetry at the protocol level, but at the product level the disparity remains a defining

constraint on the project's development trajectory.

Enterprise Users –> Headscale –> Tailscale SaaS A particularly notable interaction pattern involves enterprise and OSS-project adopters who use Headscale not primarily as an active deployment but as a strategic escape hatch. As Dalby explained, users who might otherwise be reluctant to depend on a proprietary SaaS become willing to adopt Tailscale precisely because Headscale exists as a self-hosted fallback — should Tailscale's commercial terms or availability change, a migration path remains open [4]. He cited the Incus OS project as a concrete example: its maintainer explicitly justified integrating Tailscale client support on the grounds that Headscale compatibility preserved the project's open-source ethos. This dynamic means Headscale simultaneously reduces vendor lock-in risk for its users and, paradoxically, expands Tailscale's addressable market by making the commercial product more palatable to lock-in-sensitive adopters. From Tailscale Inc.'s perspective, Headscale functions both as a community goodwill signal and as an indirect sales enabler — a dual role that further explains the company's willingness to fund its development.

Reflection The stakeholder analysis relies heavily on a single primary source — the lead maintainer's FOSDEM 2026 keynote [4] — with limited triangulation from independent sources, which may introduce perspective bias. While supplementary signals were gathered from GitHub activity and community discussions on Discord, these sources are informal and indirect in nature and therefore cannot be considered strong standalone evidence for validating the findings.

5 Software Metrics

5.1 Metric Selection

To reveal the technical debt and maintainability challenges as the project scaled without a proportional increase in maintainers or governance structure, four metrics were selected to probe different quality dimensions:

1. **Hotspot Risk**: Given that Headscale originated from informal development, early core files may have gradually accumulated complexity and potential design antipatterns. This metric helps identify whether these files continue to act as structural bottlenecks, potentially constraining future refactoring and feature development.
2. **Knowledge Concentration**: As the project is driven by a small number of core developers, critical implementation knowledge may remain centralized even as the development team expands. This metric assesses whether community growth has successfully distributed that knowledge.
3. **Coordination Latency**: Informal projects lack efficient structures. This metric reflects whether the lack of structure leads to increasing coordination overhead as the project scales.
4. **Mean Time to Repair**: This metric reflects the robustness of the project's maintenance capacity and how community growth influences the bug resolution process, indicating whether maintenance efficiency scales effectively with project growth.

5.2 Stage Window Definition

To better understand the project's evolution, the metric analysis uses a hybrid windowing strategy. M1 and M2 use cumulative windows (Start → current stage) to capture the accumulation of technical debt, while M3 and M4 use segmented windows (previous stage →

current stage) to evaluate friction in each stage. For cumulative windows, the start release version v0.1.0 is used as the lower bound.

Stage	Version	Date	Cumulative Range (M1/M2)	Segmented Range (M3/M4)
Start	v0.1.0	2021-06-14	v0.1.0	v0.1.0
Early growth	v0.12.1	2021-12-24	v0.1.0-v0.12.1	v0.1.0-v0.12.1
Expansion	v0.17.1	2022-12-05	v0.1.0-v0.17.1	v0.12.1-v0.17.1
Maturity	v0.22.3	2023-05-12	v0.1.0-v0.22.3	v0.17.1-v0.22.3
Current	v0.28.0	2026-02-04	v0.1.0-v0.28.0	v0.22.3-v0.28.0

Table 1: Selected stable releases and window definition used for metric computation.

5.3 Static Analysis

The Herfindahl–Hirschman Index (HHI), originally developed as a market concentration measure in economics [7], is here adapted as a general-purpose concentration metric to quantify the distribution of file churn across the codebase.

$$\mathrm{HHI} = \sum_i s_i^2,$$

where s_i is the touch-event share of file i in the corresponding window.

5.3.1 M1 Hotspot Concentration (HHI)

M1 uses file churn concentration to identify structural hotspots across release windows. Lower HHI and lower Top10 share indicate better distribution of change across the codebase, while higher values suggest concentration in a few files.

Stage	Touch Events	Top10 Share (%)	Top1 Share (%)	HHI	Top 3 Hotspots
Start	313	51.12	9.27	0.0376	`cmd/headscale/headscale.go`; `README.md`; `api.go`
Early growth	1322	40.47	6.35	0.0238	`app.go`; `api.go`; `machine.go`
Expansion	3455	31.43	5.62	0.0164	`app.go`; `machine.go`; `api.go`
Maturity	4436	27.28	4.62	0.0129	`app.go`; `machine.go`; `CHANGELOG.md`
Current	9314	16.89	3.47	0.0059	`CHANGELOG.md`; `app.go`; `machine.go`

Table 2: M1 Hotspot Concentration by stage.

The concentration of churn declines over the full history (Top10 share from 51.12% to 16.89%,

HHI from 0.0376 to 0.0059), suggesting broader change distribution. However, recurring core files remain visible hotspots across phases. This twist may suggest a redistribution of change pressure rather than its elimination. The project still relies on a small set of critical files that may accumulate technical debt without proper attention. In addition, notice that the CHANGELOG.md file becomes a hotspot in later stages, suggesting that documentation churn is absorbing a portion of the measured hotspot pressure, which may mask underlying code-level concentration.

5.3.2 M2 Knowledge Concentration

M2 evaluates authorship concentration at commit level. Lower HHI and higher Bus Factor [8] indicate better knowledge distribution.

Stage	Commits	Authors	Top1 Share (%)	Top2 Share (%)	HHI	Effective Authors	Bus Factor 50
Start	121	7	74.38	90.08	0.5825	1.72	1
Early growth	611	32	50.57	68.74	0.3114	3.21	1
Expansion	1740	96	42.87	64.37	0.2419	4.13	2
Maturity	2005	130	42.69	61.40	0.2333	4.29	2
Current	2803	238	48.16	61.54	0.2602	3.84	2

Table 3: M2 Knowledge Concentration by stage.

According to the tendency of the data, a structural twist is observed in the knowledge concentration pattern: knowledge concentration weakens through Maturity, then re-concentrates in Current (Top1 share rises from 42.69% to 48.16% and HHI from 0.2333 to 0.2602) while the total author count keeps rising. This phenomenon may indicate that among the 238 authors, the vast majority belong to the long tail of low-frequency contributors, and the core developers still play a significant role. This conclusion is consistent with the BusFactor 50 result, which is stable across all stages.

5.4 Dynamic Analysis

5.4.1 M3 Coordination Latency

M3 tracks cross-author handoff dynamics on the same file, where latency is measured as the median delay between adjacent edits by different authors. This operationalisation is inspired by socio-technical congruence research, where coordination requirements arise from task dependencies between developers [9], and delays in recognising or addressing these coordination needs can negatively impact collaboration efficiency [10].

Stage	Handoff Rate (%)	Median Handoff (days)	P75 Handoff (days)
Start	23.98	4.04	13.37
Early growth	30.44	6.30	20.23
Expansion	42.22	9.37	26.78
Maturity	34.08	15.31	43.57
Current	32.49	21.27	60.08

Table 4: M3 Coordination Latency by stage.

Coordination overhead grows over time: while handoff rate peaks in the middle of the period, handoff delay rises substantially. This internal divergence within the metric may suggest that

code ownership patterns are emerging. Considering the knowledge concentration pattern in M2, this phenomenon may be the observable manifestation of accumulated organisational technical debt, expressed as increasing time overhead.

5.4.2 M4 Mean Time to Repair

M4 employs two complementary sub-measurements (M4A and M4B) to mitigate the limitations of each individual proxy:

M4A File-Touch Proxy For each fix-related commit in segmented windows, M4A is measured as the delay between the fix commit timestamp and the latest previous touch timestamp on any changed file. This metric serves as a proxy for repair latency, inspired by defect resolution time analyses in mining software repositories literature [11]. While conceptually related to Mean Time to Repair (MTTR), which measures the time required to restore a system after failure [12], this study operationalises repair latency at the file and commit level.

Stage	Samples	Mean (days)	Median (days)	P75 (days)
Start	11	8.17	0.81	8.44
Early growth	93	3.12	0.86	3.19
Expansion	156	8.36	0.76	8.45
Maturity	20	15.38	2.22	24.91
Current	122	19.49	5.05	19.22

Table 5: M4A File-Touch Proxy MTTR by stage.

M4B Bug-Only Issues Given that this repository includes many feature requests issues, only bug fix issues are included in M4B to better capture the Mean Time to Repair (MTTR). For each closed bug-labeled issue in `juanfont/headscale`, M4B is measured as the time difference between `closed_at` and `created_at`.

Stage	Samples	Mean (days)	Median (days)	P75 (days)
Start	0	N/A	N/A	N/A
Early growth	5	16.00	6.87	27.91
Expansion	103	31.86	5.60	43.17
Maturity	74	64.17	12.92	82.84
Current	479	81.12	20.05	102.22

Table 6: M4B Bug-Only Issue MTTR by stage

The high mean and low median in M4A suggest a long-tail distribution of repair times, likely attributable to a small number of structurally complex, high-coupling files.

M4B shows a long-tail repair profile in the live dataset (overall median 13.84 days, P90 205.41 days), with stage-level median MTTR rising from 6.87 days in Early growth to 20.05 days in Current. Moreover, the mean MTTR rises more substantially (16.00 days to 81.12 days), which may reflect that the tech debt in current stage impacts the user's experience and confidence in the project robustness.

5.5 Metric Summary and Reflection

Summary Holistically considering all the four metrics, the analysis reveals a general picture of Headscale's technical debt evolution. In Phase 1 (Start to Early Growth), the project is characterized by strong founder dependency, with a single dominant contributor (Bus

Factor 50 = 1) and highly concentrated code hotspots in core logic. During this stage, mean time to repair (MTTR) remains low due to minimal decision latency, as the founder directly resolves issues. This pattern reflects a form of "productive" or "benign" technical debt, representing a necessary trade-off for rapid iteration. In Phase 2 (Expansion to Maturity), the number of contributors increases and the Herfindahl–Hirschman Index (HHI) declines, suggesting a superficially healthier distribution of contributions. However, coordination latency begins to rise significantly, and MTTR exhibits increasing variance. Simultaneously, code hotspots expand beyond core logic into peripheral artifacts such as changelog files, indicating a shift in system activity. At this stage, technical debt transitions from a purely technical phenomenon to an organizational one. In Phase 3 (Current), the project enters a scaling crisis marked by the simultaneous deterioration of multiple indicators: knowledge concentration as measured by M2 HHI rebounds, coordination delay (M3, P75) reaches 60 days, mean MTTR for bug-related issues (M4B) rises to 81 days, and the bug sample size expands to 479. A pronounced disparity emerges between the nominal contributor base (238 authors) and the effective decision-making core (Bus Factor 50 = 2), revealing a widening governance gap. Collectively, these patterns point to a typical open-source failure mode [13] in which apparent popularity and growth mask an underlying loss of coordination efficiency and an erosion of project control.

Reflection

- For M3: Cross-author file transitions are treated as a proxy for coordination overhead; however, such transitions may also reflect independent ownership shifts rather than active collaboration handoffs.
- For M4A: M4A measures the elapsed time since the most recent modification to affected files, which serves as a proxy for repair latency rather than a direct measure of fault introduction–to–resolution interval. The heuristic keyword-based identification of fix-related commits introduces noise, as the pattern may both miss synonymous terms and include false positives such as error-handling refactors.

6 Conclusion

The data provides clear answers for the research question and reveals the typical technical debt evolution pattern in an organically grown open-source project. Technical debt initially manifests as a concentration of code hotspots table 2, and appears to dissipate as the codebase expands. However, knowledge debt persists over time and resurges during later stages of scaling table 3, constituting the most structurally significant risk within the project. Coordination debt, in contrast, grows monotonically with increasing project complexity, manifesting as the time overhead imposed by organisational inefficiencies table 4. Finally, quality debt can be viewed as the cumulative outcome of these preceding forms of debt: although it emerges at a later stage, it exerts the most tangible impact on end users table 5 & table 6. It is reasonable to assume that such a pattern is not unique to Headscale, but rather reflects a common trajectory for organically grown open-source projects, consistent with findings in the literature [13].

References

[1] J. Font and contributors, *Frequently asked questions*, Accessed: March 2026, 2026. [Online]. Available: https://headscale.net/stable/about/faq/.

[2] J. Font and contributors, *Headscale: An open source, self-hosted implementation of the tailscale control server*, Accessed: March 2026, 2026. [Online]. Available: https://github.com/juanfont/headscale.

[3] J. Font and contributors, *About*, Accessed: March 2026, 2026. [Online]. Available: https://headscale.net/stable/#design-goal.

[4] K. Dalby, *Headscale & tailscale: The complementary open source clone*, Accessed: March 2026, 2026. [Online]. Available: https://fosdem.org/2026/schedule/event/KYQ3LL-headscale-the-complementary-open-source-clone/.

[5] *Microsoft windows*, 2025. [Online]. Available: https://support.scss.tcd.ie/wiki/Microsoft_Windows.

[6] N. Rozanski and E. Woods, *Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives*, 2nd. Addison-Wesley, 2011. [Online]. Available: %5Curl%7Bhttps://www.viewpoints-and-perspectives.info%7D.

[7] T. Q. Truong, J. Zhang, Z. Li, and L. Wang, "Integrated herfindahl–hirschman index, compromise programming, and -constraint method for multicriteria performance-based transportation budget allocation," *Transportation Research Record*, vol. 2675, no. 10, pp. 468–480, 2021. DOI: 10.1177/03611981211011648. eprint: https://doi.org/10.1177/03611981211011648. [Online]. Available: https://doi.org/10.1177/03611981211011648.

[8] Wikipedia contributors, *Bus factor*, Accessed: April 2026. [Online]. Available: https://en.wikipedia.org/wiki/Bus_factor.

[9] M. Cataldo and J. D. Herbsleb, "Coordination breakdowns and their impact on development productivity and software failures," *IEEE Transactions on Software Engineering*, vol. 39, no. 3, pp. 343–360, 2013. DOI: 10.1109/TSE.2012.32. [Online]. Available: https://herbsleb.org/web-pubs/pdfs/Cataldo-Coordination-2013.pdf.

[10] K. Blincoe, G. Valetto, and S. Goggins, "Proximity: A measure to quantify the need for developers' coordination," in *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work (CSCW)*, 2012, pp. 1351–1360. DOI: 10.1145/2145204.2145406. [Online]. Available: https://www.researchgate.net/publication/220879306_Proximity_A_measure_to_quantify_the_need_for_developers%27_coordination.

[11] S. Kim and E. J. Whitehead, "How long did it take to fix bugs?" In *Proceedings of the 2006 International Workshop on Mining Software Repositories*, ser. MSR '06, Shanghai, China: Association for Computing Machinery, 2006, pp. 173–174, ISBN: 1595933972. DOI: 10.1145/1137983.1138027. [Online]. Available: https://doi.org/10.1145/1137983.1138027.

[12] A. Perry and M. Luebbe. "Testing for reliability." Accessed: 2026-04-19. (2016), [Online]. Available: https://sre.google/sre-book/testing-reliability/.

[13] J. Coelho and M. T. Valente, "Why modern open source projects fail," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017, Paderborn, Germany: Association for Computing Machinery, 2017, pp. 186–196, ISBN: 9781450351058. DOI: 10.1145/3106237.3106246. [Online]. Available: https://doi.org/10.1145/3106237.3106246.